Day 79 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 3 reference

Watch: Batch Norm vs Layer Norm, Explained (DataMListic) | Layer Normalization in Transformers, Layer Norm Vs Batch Norm (CampusX)

`day79_layernorm_vs_batchnorm.py`, segment by segment, in the order `main()` uses things. Everything up to the encoder tagger is Day 78's code, copied verbatim (see Day 78's syntax guide); the one change is shown first. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: x = the numbers being normalised, shape (sentences, positions, features) · N = real positions in the batch · μ_f, σ_f = feature f's mean and spread over them · momentum = how much of the old running average is kept

Segment 0 — Reused code

The one change to Day 78's code

CODE

```
def __init__(self, vocab_size, n_tags, n_blocks=1, pre_ln=False, pe_every_block=True, lr=1.0,
    clip=1.0, seed=0,
             block_cls=EncoderBlock, **block_kw):
    rng = np.random.RandomState(seed)
    self.n_tags, self.pre, self.pe_every, self.lr, self.clip = n_tags, pre_ln, pe_every_block, lr,
        clip
    self.P = dict(E=rng.randn(vocab_size, D) * 0.1)
    self.blocks = [block_cls(rng, pre_ln=pre_ln, **block_kw) for _ in range(n_blocks)]
```

- `block_cls=EncoderBlock` — a class passed as an argument, with Day 78's as the default.
- `block_cls(rng, ...)` — calling whatever class was passed: NormBlock here, so LayerNorm and BatchNorm get the same weights.

fit, with the batch size as a setting

CODE

```
def fit(data, vocab_size, n_tags, seed, epochs=EPOCHS, batch_size=32, **kw):
    (ids_tr, tags_tr, mask_tr), (ids_va, tags_va, mask_va), (ids_te, tags_te, mask_te) = data
    net = NormTagger(vocab_size, n_tags, seed=seed, **kw)
    rng = np.random.RandomState(seed)
    lens = mask_tr.sum(axis=1)
    best_val, best, best_epoch = -1.0, None, -1
    for epoch in range(epochs):
        perm = rng.permutation(len(ids_tr))
        for start in range(0, len(perm), batch_size):
            idx = perm[start:start + batch_size]
            T = lens[idx].max()
            net.train_step(ids_tr[idx, :T], tags_tr[idx, :T], mask_tr[idx, :T])
        val = token_accuracy(net.predict(ids_va), tags_va, mask_va)
        if val > best_val:
            best_val, best, best_epoch = val, net.snapshot(), epoch
    net.restore(best)
```

- `net.snapshot()` — copies the weights AND BatchNorm's running averages: both define the model at test time.
- `net.predict(ids_va)` — eval mode: BatchNorm uses its running averages, as it would after training.

Segment 1 — Batch normalisation

batch_norm

MATH

$$\mu_f = (1/N)\,\Sigma_{\text{real positions}}\, x_f, \quad \sigma_f^2 = (1/N)\,\Sigma\,(x_f - \mu_f)^2, \quad y = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)} \cdot g + b$$

CODE

```
def batch_norm(x, g, b, mask, eps=1e-5):
    real = x[mask]
    mu, var = real.mean(axis=0), real.var(axis=0)
    sd = np.sqrt(var + eps)
    xhat = (x - mu) / sd
    return xhat * g + b, (xhat, sd, mask), mu, var
```

- `x[mask]` — boolean indexing with a (sentences, positions) mask: an (N, features) array of the real positions.
- `real.mean(axis=0)` — one mean per feature, over every real position of the batch.
- `(x - mu) / sd` — broadcasting: padded positions are normalised with the same statistics, but did not shape them.

batch_norm_backward

MATH

$$z = (x - \mu) / \sigma,\ u = dy \cdot g: \quad dx_i = (\,u_i - [i\ real] \cdot (mean(u) + z_i\ mean(u\ z))\,) / \sigma, \quad \text{the means taken over } N$$

CODE

```
def batch_norm_backward(dy, g, cache):
    xhat, sd, mask = cache
    d, N = xhat.shape[-1], mask.sum()
    dxhat = dy * g
    mean_d = dxhat.reshape(-1, d).sum(axis=0) / N
    mean_dx = (dxhat * xhat).reshape(-1, d).sum(axis=0) / N
    dx = (dxhat - mask[..., None] * (mean_d + xhat * mean_dx)) / sd
    return dx, (dy * xhat).reshape(-1, d).sum(axis=0), dy.reshape(-1, d).sum(axis=0)
```

- `xhat.shape[-1], mask.sum()` — features, and N, the number of real positions (True counts as 1).
- `mask[..., None] * (...)` — the correction through the mean and spread applies only to the positions that shaped them.
- `dxhat.reshape(-1, d).sum(axis=0) / N` — sums over every position that USED the statistics, divided by the N that MADE them.

NormBlock: the norm as a setting

MATH

$$\text{running} \leftarrow 0.9 \cdot \text{running} + 0.1 \cdot \text{this batch's statistic}$$

CODE

```
class NormBlock(EncoderBlock):
    def __init__(self, rng, kind="layer", momentum=0.9, **kw):
        super().__init__(rng, **kw)
        self.kind, self.momentum = kind, momentum
        self.training, self.update_running = True, True
        self.running = {w: (np.zeros(D), np.ones(D)) for w in ("1", "2")}
        self.pos_mask = None
    def _norm(self, x, which):
        if self.kind == "layer":
            return super()._norm(x, which)
        g, b = self.P["g" + which], self.P["b" + which]
        mask = self.pos_mask if self.kind == "batch" else np.ones(x.shape[:2], bool)
        if not self.training:
            mu, var = self.running[which]
            return (x - mu) / np.sqrt(var + 1e-5) * g + b, None
        y, cache, mu, var = batch_norm(x, g, b, mask)
        if self.update_running:
            rm, rv = self.running[which]
            self.running[which] = (self.momentum * rm + (1 - self.momentum) * mu, self.momentum *
                rv + (1 - self.momentum) * var)
        return y, cache
    def _norm_back(self, dy, which, cache, g):
        if self.kind == "layer":
            return super()._norm_back(dy, which, cache, g)
        dx, g["g" + which], g["b" + which] = batch_norm_backward(dy, self.P["g" + which], cache)
        return dx
```

- `class NormBlock(EncoderBlock)` — inherits Day 78's block; only _norm and _norm_back are redefined.
- `super()._norm(x, which)` — the LayerNorm path is Day 78's code, unchanged.
- `self.running[which] = (m * rm + (1 - m) * mu, ...)` — an exponential moving average: 90% old, 10% this batch.
- `if not self.training: ... return ..., None` — eval mode uses the running averages and needs no cache.

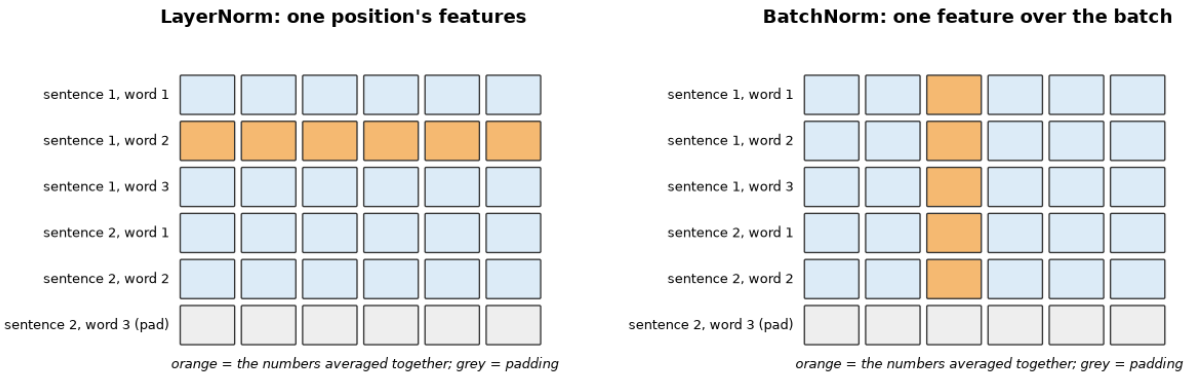
NormTagger

CODE

```
class NormTagger(EncoderTagger):
    def __init__(self, vocab_size, n_tags, **kw):
        super().__init__(vocab_size, n_tags, block_cls=NormBlock, **kw)
    def set_mode(self, training, update_running=True):
        for b in self.blocks:
            b.training, b.update_running = training, update_running
    def forward(self, ids):
        for b in self.blocks:
            b.pos_mask = ids != PAD
        return super().forward(ids)
    def predict(self, ids, batch_stats=False):
        self.set_mode(training=batch_stats, update_running=False)
        out = super().predict(ids)
        self.set_mode(training=True)
        return out
    def snapshot(self):
        return copy.deepcopy(self.params()), [copy.deepcopy(b.running) for b in self.blocks]
    def restore(self, snap):
        P, running = snap
        for k, v in P.items():
            self.params()[k][...] = v
        for b, r in zip(self.blocks, running):
            b.running = r
```

- `super().__init__(..., block_cls=NormBlock, **kw)` — Day 78's tagger, built from NormBlocks; kind=... travels in **kw to each block.
- `b.pos_mask = ids != PAD` — every block learns which positions are real before the forward pass.
- `predict(self, ids, batch_stats=False)` — a keyword argument with a default: Section 7 asks for batch statistics at test time.
- `for b, r in zip(self.blocks, running)` — pairs each block with its saved running averages.

SYSTEM DIAGRAM



Which numbers each norm averages over.

Segment 2 — Experiments

demo_batch_dependence: two sets of companions

CODE

```
for training in (True, False):
    net.set_mode(training, update_running=False)
    outs = []
    for comp in (others[:7], others[7:]):
        b = ids[np.concatenate([[0], comp])]
        T = (b != PAD).sum(axis=1).max(); n0 = (b[0] != PAD).sum()
        outs.append(net.forward(b[:, :T]).reshape(len(b), T, -1)[0, :n0])
    row.append(np.abs(outs[0] - outs[1]).max())
```

- `np.concatenate([[0], comp])` — sentence 0 first, then the companions.
- `net.forward(...).reshape(len(b), T, -1)[0, :n0]` — sentence 0's real words only, from a (sentences x positions, tags) array.

demo_padding: 20 extra padding positions

CODE

```
wide = np.zeros((8, T + 20), dtype=int); wide[:, :T] = ids[:, :T]
real = (ids[:, :T] != PAD).ravel()
print(f"\n{'norm':<28}{'largest change':>16}")
for kind, label in [("layer", "LayerNorm"), ("batch", "BatchNorm (padding left out)"),
    ("batch_all", "BatchNorm (padding counted)")]:
    net = NormTagger(len(itos), len(tag_names), kind=kind, seed=0)
    net.set_mode(True, update_running=False)
    a = net.forward(ids[:, :T])
    b = net.forward(wide).reshape(8, T + 20, -1)[:, :T].reshape(8 * T, -1)
    print(f"{label:<28}{np.abs(a - b)[real].max():>16.2e}")
```

- `np.zeros((8, T + 20), dtype=int)` — a wider array of padding ids (0), with the sentences copied into its left part.
- `[:, :T].reshape(8 * T, -1)` — cut the wide output back to the narrow layout, so the same words line up.

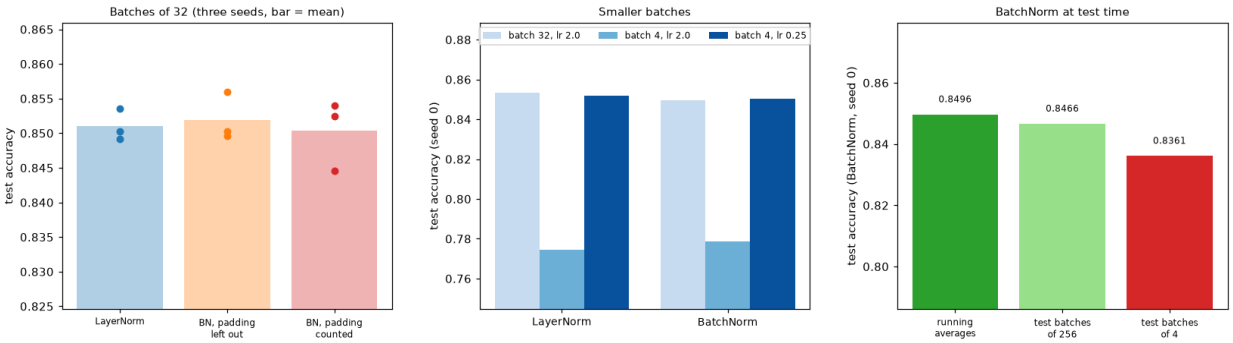
demo_train_vs_eval: three ways to normalise at test time

CODE

```
run = token_accuracy(net.predict(ids), tags, mask)
b256 = token_accuracy(net.predict(ids, batch_stats=True), tags, mask)
pred4 = np.zeros(ids.shape, dtype=int)
net.set_mode(True, update_running=False)
for s in range(0, len(ids), 4):
    b = ids[s:s + 4]; T = (b != PAD).sum(axis=1).max()
    pred4[s:s + 4, :T] = net.forward(b[:, :T]).argmax(axis=1).reshape(len(b), T)
net.set_mode(True)
b4 = token_accuracy(pred4, tags, mask)
```

- `net.predict(ids, batch_stats=True)` — each batch of 256 test sentences normalised with its own statistics.
- `net.set_mode(True, update_running=False)` — batch statistics without touching the running averages.

REAL GRAPH



Real graph from the script: batches of 32, small batches, BatchNorm at test time.

Quick Syntax Glossary — new or reinforced constructs

- `def f(..., block_cls=EncoderBlock)` — a class as a default argument value.
- `x[mask]` — boolean indexing over the first two axes: the selected rows, flattened.
- `mask[..., None]` — add a trailing axis so a (m, T) mask multiplies a (m, T, d) array.
- `class B(A): ... super().f(...)` — redefine a method and still call the original.
- `zip(a, b)` — walk two lists in step.
- `a = b if cond else c` — a conditional expression.

End of Day 79 syntax reference. For the experiments and what they showed, see the Day 79 complete guide.